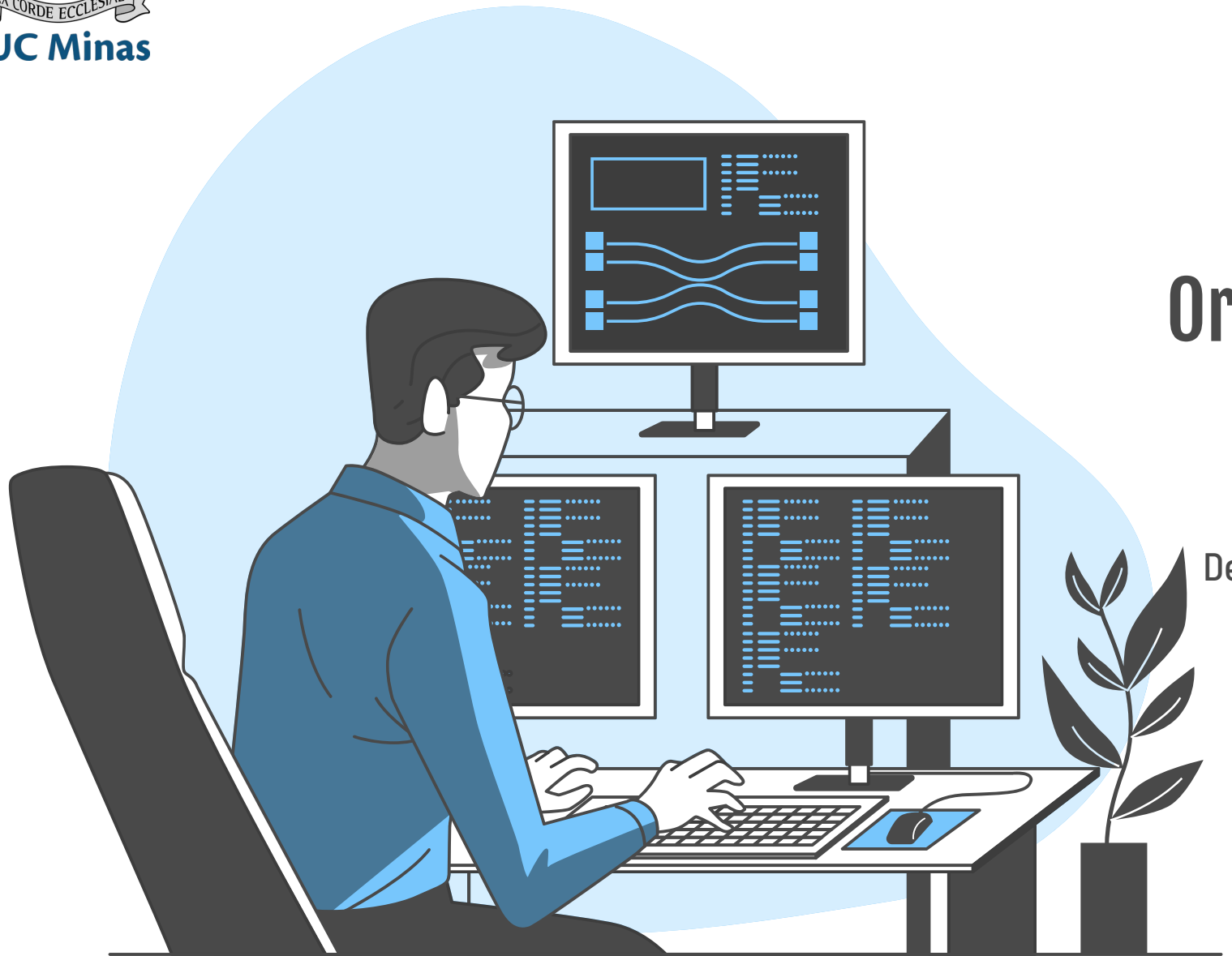
PUC Minas



Aula 02 · FullStack

Organizando o repositório: pastas Back e Front

Desenvolvimento e Integração de Aplicações Web

Prof. Dr. João Paulo Aramuni

Engenharia de Software · PUC Minas

github.com/joaopauloaramuni/desenvolvimento-e-integracao-de-aplicacoes-web

Apresentação da aula

Nesta aula você vai

1. Montar a estrutura **monorepo** com `Back/` e `Front/` lado a lado
2. Criar cada parte com o comando certo, na pasta certa
3. Trabalhar com as duas partes no VS Code (terminais divididos e *workspace*)
4. Escrever o `.gitignore` de cada parte e um da raiz
5. Documentar portas, variáveis e comandos no README

Antes de começar

- Aula 01 desta pasta
- Git instalado e conta no GitHub

Duração sugerida

100 min

Código da disciplina

Projetos de referência na pasta PROJETOS do GitHub

Roteiro

01 A estrutura dos projetos da disciplina

02 Criando do zero

03 Trabalhando com as duas partes no VS Code

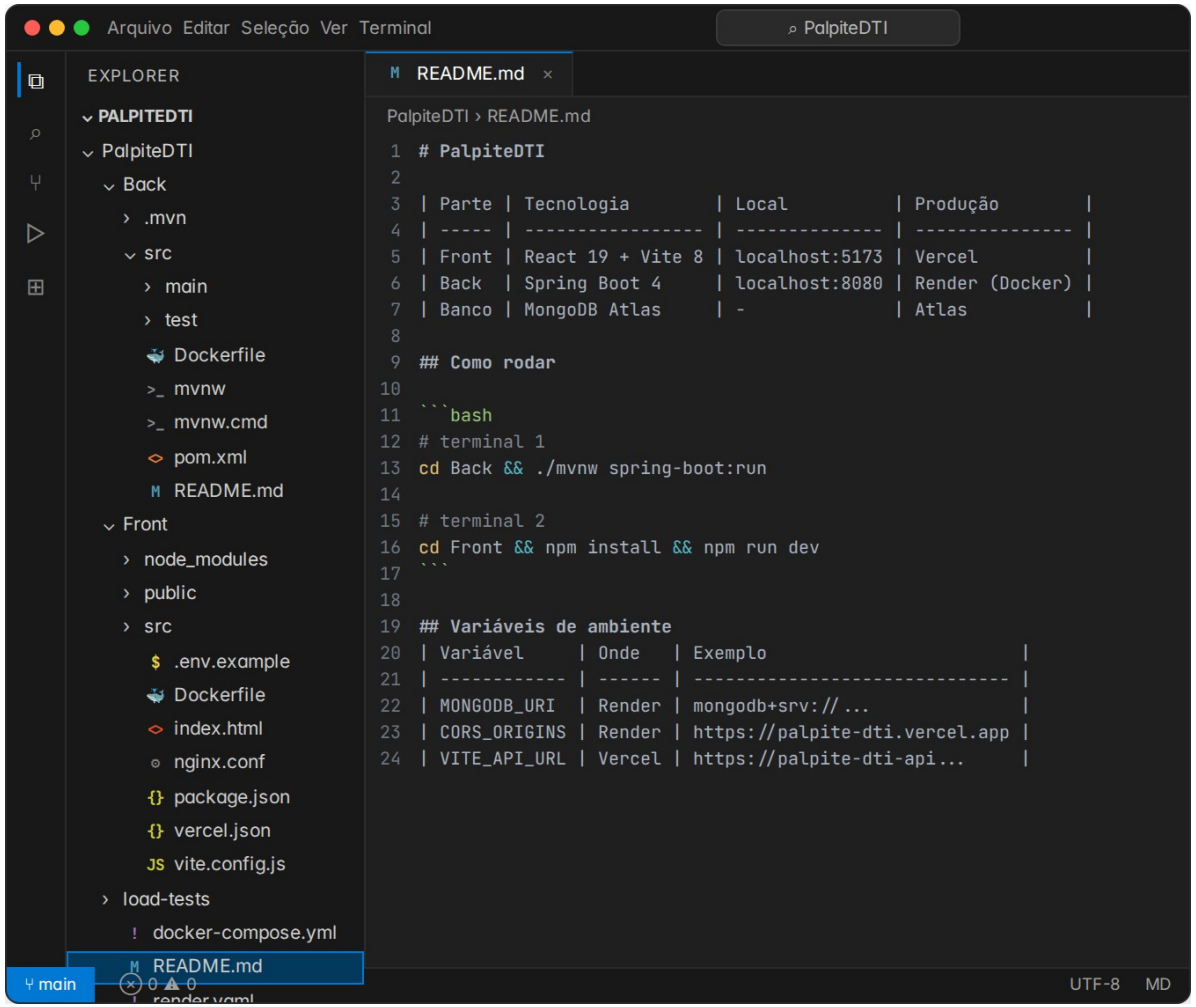
04 .gitignore: o que nunca sobe

05 Aprofundando: Git em equipe

01

A estrutura dos projetos da disciplina

A estrutura dos projetos da disciplina



Todos os projetos da pasta `PROJETOS/FullStack` seguem o mesmo formato: **um repositório**, **duas pastas**. O Vercel e o Render conseguem publicar cada pasta separadamente usando a opção *Root Directory*.

Estrutura do `PalpiteDTI`: `Back/` e `Front/` na raiz, com os arquivos de deploy de cada um.

A estrutura dos projetos da disciplina

Arquivo na raiz	Para que serve
`README.md`	Visão geral, portas, comandos e variáveis (o “manual” do projeto)
`docker-compose.yml`	Sobe front, back e banco juntos com Docker (Deploy, aula 04)
`render.yaml`	*Blueprint*: o Render cria o serviço do back lendo este arquivo (Deploy, aula 06)
`.gitignore`	Regras comuns (pode haver também um em cada pasta)

02

Criando do zero

Criando do zero

Terminal

```
aluno@pc:~/biblioteca$ mkdir biblioteca && cd biblioteca
aluno@pc:~/biblioteca$ git init
Initialized empty Git repository in /home/aluno/biblioteca/.git/
# 1) Back: gere no start.spring.io com Artifact = Back, extraia aqui dentro
aluno@pc:~/biblioteca$ unzip ~/Downloads/Back.zip -d .
# 2) Front: Vite com o nome da pasta = Front
aluno@pc:~/biblioteca$ npm create vite@latest Front -- --template react
aluno@pc:~/biblioteca$ cd Front && npm install && cd ..
aluno@pc:~/biblioteca$ ls
Back  Front
```

Montando o monorepo: o Back vem do Spring Initializr e o Front do `create-vite`, ambos dentro da mesma pasta.

Criando do zero

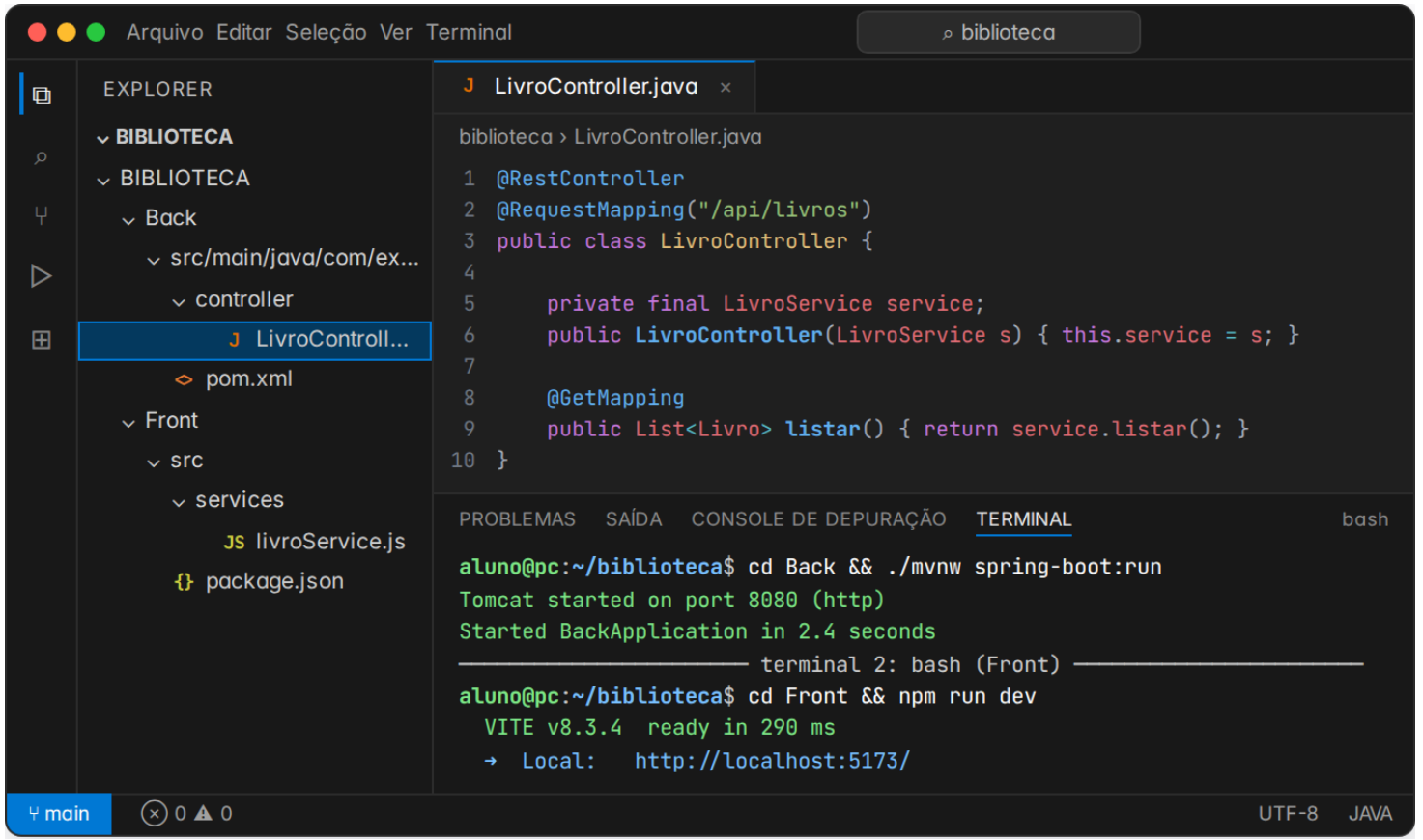
Dica

O Initializr gera o `.zip` com uma pasta chamada pelo *Artifact*. Use `Back` como Artifact e o nome já fica certo. No Vite, o nome da pasta é o argumento do comando (`Front`). Mantenha esses dois nomes: os scripts e os exemplos de deploy contam com eles.

03

Trabalhando com as duas partes no VS Code

Trabalhando com as duas partes no VS Code



Abra a **pasta raiz** (`code biblioteca`). Você verá `Back/` e `Front/` no Explorer. Para rodar as duas partes, use **dois terminais**: clique no **+** do painel de terminal ou no ícone de **dividir** (`Ctrl+Shift+5`).

Raiz aberta no VS Code. Um terminal roda o Spring (pasta `Back`) e outro o Vite (pasta `Front`).

Workspace com várias raízes (opcional)

As extensões de Java às vezes demoram para reconhecer um projeto Maven que não está na raiz. Se isso acontecer, use **Arquivo → Adicionar Pasta ao Workspace** para adicionar `Back` e `Front` como raízes separadas e salve com **Salvar Workspace Como...**:

biblioteca.code-workspace

```
{
  "folders": [
    { "path": "Back",  "name": "☕ Back (Spring Boot)" },
    { "path": "Front", "name": "⚛️ Front (React + Vite)" },
    { "path": ".",     "name": "📁 Raiz" }
  ]
}
```

04

.gitignore: o que nunca sobe

.gitignore: o que nunca sobe

.gitignore (na raiz)

```
# ---------- Back (Java / Maven) ----------
Back/target/
Back/.idea/
*.iml

# ---------- Front (Node / Vite) ----------
Front/node_modules/
Front/dist/
Front/.env.local
Front/.env.*.local

# ---------- Geral ----------
.env
.DS_Store
.vscode/
```

.gitignore: o que nunca sobe

Atenção

Antes do primeiro `git push`, rode `git status` e confira a lista. Se aparecer `node_modules/`, `target/`, `.env.local` ou um `application.properties` com senha real, **pare** e corrija o `.gitignore`. Segredo que foi para o GitHub precisa ser considerado vazado: troque a senha no Atlas/Supabase.

.gitignore: o que nunca sobe

Primeiro envio ao GitHub

```
git add .
git status                          # confira!
git commit -m "Estrutura inicial: Back (Spring Boot) e Front (React + Vite)"
git branch -M main
git remote add origin https://github.com/seu-usuario/biblioteca.git
git push -u origin main
```

05

Aprofundando: Git em equipe

Aprofundando: Git em equipe

Comando	Para que serve
`git switch -c feature/busca`	Cria uma branch para a tarefa
`git add -p`	Escolhe trecho por trecho o que entra no commit
`git commit -m "Back: adiciona filtro por nome"`	Commit com mensagem que diz a parte e a mudança
`git pull --rebase`	Traz as mudanças dos colegas antes de enviar
`git push -u origin feature/busca`	Publica a branch
Pull request no GitHub	Revisão antes de juntar na `main`

Monorepo x dois repositórios

Monorepo (Back/ e Front/)

- Uma mudança de contrato em um commit só
- Um README para tudo
- Padrão da disciplina

vs

Dois repositórios

- Permissões e histórico separados
- Equipes independentes
- Precisa sincronizar versões do contrato

Aprofundando: Git em equipe

Mensagens de commit no padrão Conventional Commits

```
feat(back): adiciona GET /api/alunos?nome=
fix(front): mostra mensagem quando o back está offline
docs: atualiza tabela de variáveis no README
test(back): cobre DELETE com id inexistente
chore: atualiza dependências do Vite
```

Arquivos de configuração na raiz

.editorconfig

Mesma indentação e fim de linha para todos os editores.

.gitattributes

`* text=auto` evita conflitos de fim de linha entre Windows e Linux.

.vscode/extensions.json

Recomenda as extensões do projeto para quem abrir a pasta.

LICENSE

Diz como outras pessoas podem usar o código (os projetos usam MIT).

Aprofundando: Git em equipe

.vscode/extensions.json

```
{
  "recommendations": [
    "vscjava.vscode-java-pack",
    "vmware.vscode-boot-dev-pack",
    "dsznajder.es7-react-js-snippets",
    "ms-azuretools.vscode-containers"
  ]
}
```

Aprofundando: Git em equipe

Erro comum

`fatal: refusing to merge unrelated histories` ou um `node_modules` gigante no primeiro push são os dois erros mais comuns ao montar o monorepo: o primeiro acontece ao criar o repositório no GitHub *com* README e tentar enviar outro histórico (use `git pull --allow-unrelated-histories` uma vez); o segundo, por esquecer o `.gitignore` antes do primeiro `git add .`.

Aprofundando: Git em equipe

1. Crie o repositório vazio no GitHub (sem README).
2. Monte `Back/` e `Front/` localmente e escreva o `.gitignore` da raiz.
3. `git add .` e confira com `git status`.
4. Faça o primeiro commit e o `git push -u origin main`.
5. Convide os colegas em *Settings* → *Collaborators*.

06

Quiz de revisão

Quiz · questão 1 de 5

Pergunta

Qual a estrutura dos projetos FullStack da disciplina?

A Dois repositórios

B Um repositório com Back/ e Front/

C Tudo em src/

D Só Front/

Resposta

Qual a estrutura dos projetos FullStack da disciplina?

A Dois repositórios

✓ Um repositório com Back/ e Front/

C Tudo em src/

D Só Front/

Por quê: Vercel e Render usam o Root Directory para escolher a pasta.

Pergunta

Qual pasta não vai para o Git?

A Back/src

B Front/src

C Back/target

D Front/public

Resposta

Qual pasta não vai para o Git?

A Back/src

B Front/src

✓ Back/target

D Front/public

Por quê: target/ e node_modules/ são gerados.

Pergunta

Como criar o Front já com o nome certo?

A npm init

B npm create vite@latest Front -- --template react

C mkdir Front

D npx front

Resposta

Como criar o Front já com o nome certo?

A npm init

✓ npm create vite@latest Front -- --template react

C mkdir Front

D npx front

Por quê: O nome da pasta é o argumento do comando.

Quiz · questão 4 de 5

Pergunta

O que fazer antes do primeiro `git push`?

A Nada

B Conferir o `git status` para não enviar segredos e pastas geradas

C Apagar o README

D Rodar o build no Vercel

Resposta

O que fazer antes do primeiro `git push`?

A. Nada

✓ Conferir o `git status` para não enviar segredos e pastas geradas

C. Apagar o README

D. Rodar o build no Vercel

Por quê: Segredo enviado ao GitHub deve ser considerado vazado.

Quiz · questão 5 de 5

Pergunta

Para que serve um arquivo `.code-workspace`?

A Deploy

B Abrir Back e Front como raízes separadas no VS Code

C Configurar o CORS

D Rodar testes

Resposta

Para que serve um arquivo `.code-workspace`?

A Deploy

✓ Abrir Back e Front como raízes separadas no VS Code

C Configurar o CORS

D Rodar testes

Por quê: Ajuda as extensões Java a reconhecerem o projeto Maven.

Fechamento

Resumo da aula

- ✓ Um repositório, duas pastas: `Back/` (Spring Boot) e `Front/` (React + Vite).
- ✓ Crie o Back pelo Initializr com Artifact `Back` e o Front com `npm create vite@latest Front -- --template react`.
- ✓ No VS Code, abra a raiz e use dois terminais (ou um workspace com várias raízes).
- ✓ `.gitignore` exclui `target/`, `node_modules/`, `dist/` e os `.env.local`.
- ✓ README na raiz com portas, comandos e variáveis de ambiente.

Exercícios

1. Monte o repositório `biblioteca` com Back e Front e publique no GitHub.

2. Escreva o README com a tabela de portas, comandos e variáveis.

3. Faça `git status` e prove que nenhum arquivo proibido está sendo versionado.

4. Clone seu repositório em outra pasta e rode as duas partes só seguindo o README.

Para ir além

- Projetos `FullStack/PalpiteDTI` e `FullStack/DronesDTI` (estrutura completa com Docker e testes de carga)

Repositório da disciplina

Todos os projetos citados estão na pasta PROJETOS

Abrir no GitHub

Links e versões citados podem ter mudado. Confira sempre a documentação oficial.



Obrigado!

Dúvidas sobre a aula 02?

Prof. Dr. João Paulo Aramuni
Engenharia de Software · PUC Minas
Desenvolvimento e Integração de Aplicações Web

GitHub da disciplina
https://github.com/joaopauloaramuni/desenvolvimento-e-integracao-de-aplicacoes-web